

Agentic Proving for Program Verification

[Alessandro Sosso](#) Akhil Arora Bas Spitters

Department of Computer Science, Aarhus University

31 August 2026



Introduction

LLMs hallucinate, sometimes producing *coherent* yet *logically invalid* arguments. **Theorem provers** provide a rigorous framework to evaluate outputs. Combining **LLM + Formal frameworks** is showing *promising results*.

How do these advances translate to formal program generation and verification?

Comparison	<u>Comparative testing</u> of Lean 4 provers on CLEVER / VERINA	→ 🏆 agentic provers
Benchmarking	Tested agentic proving on <u>end-to-end</u> CLEVER pipeline	→ ✅ up to 98.1% e2e pass rate
Auditing	Systematic audit of CLEVER entries and <u>evaluation method</u>	→ !! 81 / 161 flawed entries

Key findings

Agentic is the new **SoTA**, also able to provide high-quality feedback and identify *failure causes/lingering bugs*
Current benchmarks are being *outgrown*: need for more rigorous, bug-resilient evaluation strategies.

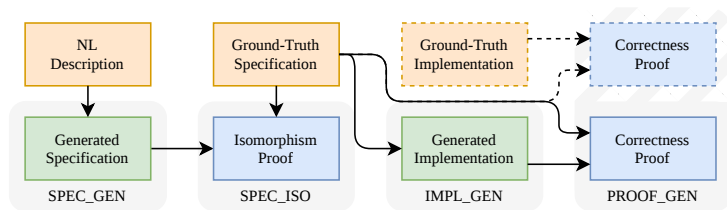
Vericoding benchmarks

CLEVER [1]

- **161** problems adapted from HumanEval
- Designed against “spec-as-code” shortcuts
- Implementations only for 49 entries, no proofs

VERINA [2]

- **189 problems** (Dafny benchmarks, LeetCode...)
- Two tiers: *basic* (108) / *advanced* (81)
- Proofs only for 46 entries



Comparison scope

Focusing on **proof generation** from ground-truth SPEC & IMPL without NL description

Lean 4 provers

Agentic Provers

- **Claude Code** [3] + `lean4-skills` [4] (domain-specific knowledge) + `lean-lsp-mcp` [5] (Lean tools access)
- **Numina-Lean-Agent** [6]: wrapper to Claude Code w/ Skills + *informal prover, discussion partner...*

Monte-Carlo Graph Search

- **Aristotle** [7]: LLM-based proof-state-aware search over a hypertree of *proving steps* and *tactics*

Whole-proof generation models

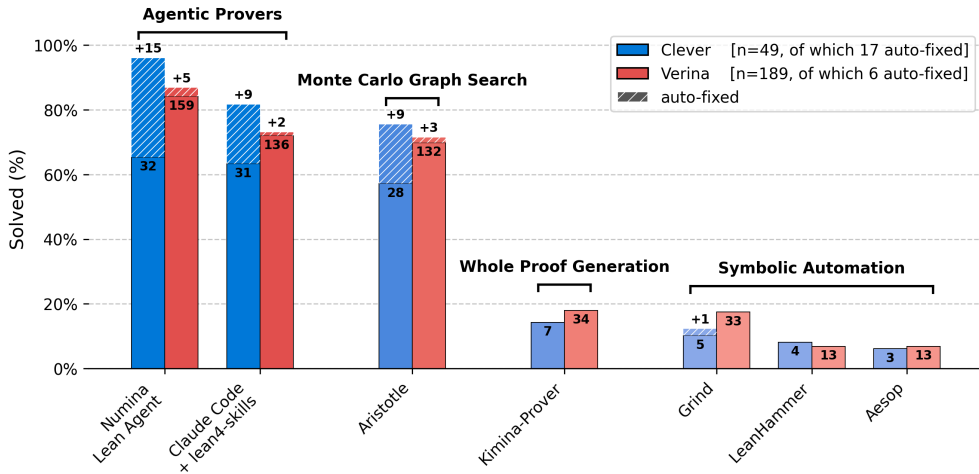
- **Kimina Prover 72B** [8]: specialized deep learning model trained / fine-tuned for *formal verification*

Baseline symbolic provers

- **Grind** [9] | **LeanHammer** [10] | **Aesop** [11] | **Canonical** [12]

Results

Agentic > MCGS >> Whole-proof Gen ~ Symbolic



Access to **compiler feedback** explains the performance gap

Bug discovery

Incorrect specifications or **implementations** were also autonomously identified

Claude through *natural language* answers, often proposing fixes and proving them

Aristotle through its specialized *goal negation search* mechanism

Bug type	CLEVER	VERINA	Total
Wrong specification logic	7	2	9
Implementation bugs	4	2	6
Missing preconditions	3	2	5
Off-by-one / boundary	3	0	3
Total	17	6	23

Next Step audit the **full CLEVER pipeline** and its native evaluation harness

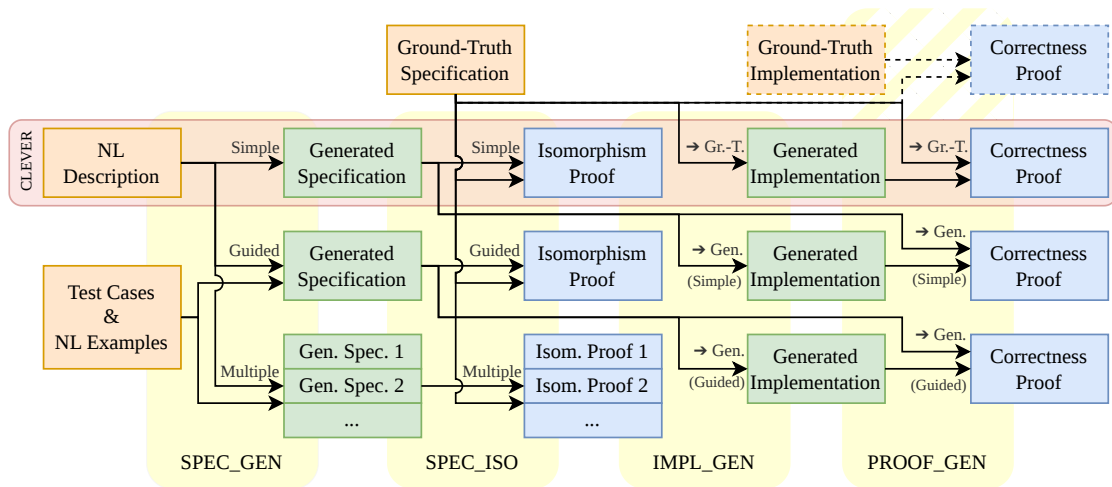
CLEVER pipeline

Specification certification

$\text{SPEC_GEN} \rightarrow \text{SPEC_ISO}$

Implementation certification

$\text{IMPL_GEN} \rightarrow \text{PROOF_GEN}$



End-to-End code generation and verification

Setup

- Claude Agent SDK (Opus 4.6) + `lean4-skills`
- System instructions + `/lean4:autoprove`

Prompt regimes

- Simple** task details, I/O format, behavior rules
- Guided** guidelines for formalization: relational formulations, patterns, precondition policy...
- Multiple** multiple spec candidates (usually 2–4)

Flag system

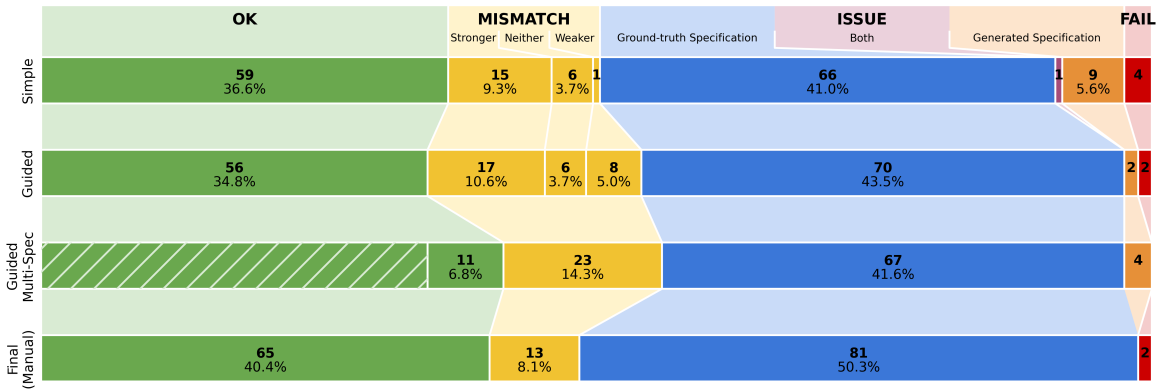
- | | |
|----------|--|
| OK | task <u>completed</u> |
| FAIL | <u>unable</u> to complete |
| ISSUE | impossible, assumptions are <u>wrong</u> |
| MISMATCH | impossible, yet assumptions are <u>valid</u> |

Flags are reliable, as confirmed by:

- manual review
- cross-checks against the benchmark harness
- consistency across runs / tasks / provers

Claude was also used to **identify the source** of the errors and **categorize** the **ISSUE** and **MISMATCH** flags

Specification certification

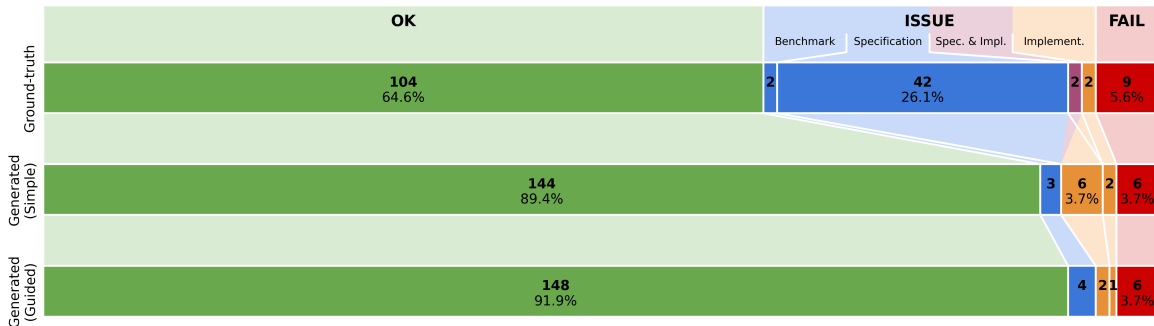


🎯 98.8% of problems with ≥ 1 **valid specification** generated

⚠️ 80 / 161 problems with a **ground-truth spec** **ISSUE** flagged at least once

⚙️ **High rate** of **MISMATCH** → evaluation has **severe limitations**

Implementation certification



✅ $70 / 80 = 87.5\%$ successfully **certified implementations** from correct ground-truth specs

🚀 $154 / 157 = 98.1\%$ arguably valid spec, implementation and proof generated

👉 **Consistency** on specification **ISSUE** → 42 in PROOF_GEN $\subset$ 81 in SPEC_ISO

Specification bugs taxonomy

Claude built a **12-type taxonomy** of specification bugs from every spec **ISSUE** at SPEC_ISO & PROOF_GEN time

Specification bug	Ground-truth specs # problems (# attempts)	Generated specs # problems (# attempts)	Example
Lean-encoding pitfalls	48 (220)	2 (2)	
Conjunct used as guard	 15 (66)		$(P \wedge Q)$ for $(P \rightarrow Q)$
Precedence bug	 12 (53)		$a \wedge b \rightarrow c$, no parentheses
$\forall$ over an invalid domain	 7 (34)		$\forall n : \mathbb{Z}, \text{length} = n$
Nat truncation	 4 (15)		$a - b = 0$ whenever $b > a$
Out-of-bounds default	 4 (15)		$\text{xs}[i] \neq 0$ past the end
Type-system mismatch	 3 (16)	 1 (1)	Nat / Int coercion
Vacuous disjunct	 3 (13)		$\dots \vee \text{result} = \text{none}$
Trivial existential	 2 (8)	 1 (1)	$\exists! S, S = 0$
Semantic errors	34 (133)	33 (57)	
Wrong semantics	 21 (90)	 18 (23)	midrange instead of median
Missing constraint	 8 (20)	 21 (29)	output order left unconstrained
Missing edge case	 4 (14)		$n = 0$, empty list
Extra constraint	 2 (9)	 3 (5)	tolerance too tight

 **Humans** trip on the Lean encoding

 **Agents** trip on the intent

Mismatches and implementation bugs taxonomy

MISMATCH type	# problems (# attempts)	Example
Precondition gap	 24 (49)	$n < 0$ left unconstrained
Edge-case behaviour	 12 (24)	$n = 0$, empty list, singleton
Ordering vs. multiset	 9 (26)	sorted list vs. any permutation
Structural choice	 7 (17)	split vs. splitOn + filter
Canonical form	 5 (11)	no leading zeros
Type-system semantics	 5 (5)	<code>Int.mod</code> vs. <code>Nat.mod</code>
Algorithmic variant	 4 (5)	binary vs. decimal digit sum
Quantifier scope	 2 (4)	distinct values vs. distinct indices
Adjacency	 1 (1)	consecutive vs. any indices

👉 **52** problems, mostly on input-domain boundary → **authoring choices**, not errors

👉 Generated specs usually stronger than ground truth → **65** attempts vs **28** weaker (32 neither)

Implementation bug	# prob. (# att.)	Example
Bounded fuel	 2 (3)	200 Newton steps
Off-by-one	 2 (2)	index or range
Wrong algorithm	 1 (1)	sort by value, not digits
Missing case	 1 (1)	$f'(x) = 0$ in Newton

👉 Implementations rarely bugged: **7 / 483** runs

Benchmark	# prob. (# att.)	Example
<i>Bugs in fixed parts</i>		
Undecided theorem	 2 (5)	Collatz, Fibonacci primes
Broken helper	 1 (3)	<code>applyOp</code> on $\div 0$
Omitted precondition	 1 (1)	roots forced into <code>Rat</code>

!👉 **4** structurally flawed entries

Cross-prover replication

Stacks **Claude Agent SDK** + Opus 4.6 | **Codex CLI** + GPT-5.4 | **OpenHands** + Kimi K2.5

Scope 161 entries $\times (2 \times \text{SPEC_ISO} + 3 \times \text{PROOF_GEN}) = 805$ cells per stack

Stack	G.T. bug confirmed of 81 flagged entries	False alarms of 80 clean entries	Cohen's κ	FAIL of 805 cells
Opus 4.6	<i>audit</i>	0	—	3.4%
GPT-5.4	76 / 81 (93.8%)	8	0.84	7.3%
Kimi K2.5	62 / 81 (76.5%)	1	0.75	20.9%
GPT $\cup$ Kimi	77 / 81 (95.1%)	8	0.85	—

✅ **No reversal:** no stack ever certified a flagged spec

🚧 Misses and false alarms sit at the **MISMATCH** boundary

🦉 Stacks differ in **failure rates**, not verdicts → the bugs belong to the **benchmark**

Lessons for benchmark design

Autoformalization is intent formalization [13]: choose the right *strength*, commit to one *interpretation*.

What should the next benchmarks look like?

Less buggy

Remove the confounders from the evaluation signal

-  Error taxonomy doubles as a checklist
-  Lint specs at authoring time: PBT, agentic judge...

81/161 **ISSUE**
greatly due to
encoding errors

More principled

Isomorphism-to-ground-truth alone is a brittle evaluator

-  Disambiguate intent: precise NL, fixed signatures, LLM-judging
-  Combine evaluators: agentic judge $\times$ PBT $\times$ proof synthesis...

52 entries
with ≥ 1
MISMATCH

More challenging

Agentic provers are no longer stressed

-  Harder sources: richer data structures...
-  Longer horizons: repo-level code...

near-ceiling
performance
few FAIL

Thank you!

References

- [1] Amitayush Thakur et al. *CLEVER: A Curated Benchmark for Formally Verified Code Generation*. 2025. arXiv: 2505.13938.
- [2] Zhe Ye et al. *VERINA: Benchmarking Verifiable Code Generation*. 2025. arXiv: 2505.23135.
- [3] Anthropic. *Claude-Agent-Sdk-Python*. *GitHub*. anthropics/claude-agent-sdk-python. 2026.
- [4] Cameron Freer. *Lean4-Skills*. *GitHub*. cameronfreer/lean4-skills. 2026.
- [5] Oliver Dressler. *Lean-Lsp-Mcp*. *GitHub*. o0o0o0o/lean-lsp-mcp. 2026.
- [6] Junqi Liu et al. *Numina-Lean-Agent: An Open and General Agentic Reasoning System for Formal Mathematics*. 2026. arXiv: 2601.14027.
- [7] Tudor Achim et al. *Aristotle: IMO-level Automated Theorem Proving*. 2025. arXiv: 2510.01346.
- [8] Haiming Wang et al. *Kimina-Prover Preview: Towards Large Formal Reasoning Models with Reinforcement Learning*. 2025. arXiv: 2504.11354.
- [9] The Lean Developers. *The Lean Language Reference: The grind tactic*. Lean FRO. 2024. URL: <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [10] Joshua Clune. *LeanHammer*. *GitHub*. JOSHCLUNE/LeanHammer. 2026.

References

- [11] Jannis Limperg and Asta Halkjær From. “Aesop: White-Box Best-First Proof Search for Lean”. In: *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP 2023. 2023.
- [12] Chase Norman and Jeremy Avigad. *Canonical for Automated Theorem Proving in Lean*. 2025. arXiv: 2504.06239.
- [13] Shuvendu K. Lahiri. *Intent Formalization: A Grand Challenge for Reliable Coding in the Age of AI Agents*. 2026. arXiv: 2603.17150 [cs].